

01-WatchBasics

基礎用法與「立即執行」機制

- 展示了 watch 的基本公式：當資料變動時，可以同時拿到「新值（newVal）」與「舊值（oldVal）」。
- 引入了配置項 { immediate: true }：預設情況下，watch 要等資料「第一次變動」才執行；加了這個參數，可以讓畫面一載入（掛載時）就**強制先執行一次**，此時舊值會是 undefined。

02-WatchDeep

物件監聽的陷阱與「Getter 函式」的解法

- **反例（Shallow）**：說明用 Getter 回傳物件時，Vue 預設只看「記憶體參照」。只改內部屬性（如 age）不會觸發，除非把整個物件換掉。
- **正例（Deep）**：展示加上 { deep: true } 後，可以強迫 Vue 深入監聽物件內部的所有欄位，但代價是新舊值會指向同一個物件（newVal === oldVal 成立）。
- **最佳實踐（Getter 精準監聽）**：展示了最聰明、效能最好的寫法。如果只關心物件裡的某個字串（如 name），就直接用 Getter 函式指向該欄位 () => state.profile.name，既不浪費效能做深層監聽，又能完美拿到新舊值。

03-WatchMultipleSources

同時監聽多個數據源

- 展示了「陣列寫法」：watch 的第一個參數可以傳入一個陣列 [firstName, lastName]。
- 對應的 Callback 函式也會以「陣列對齊」的方式，同時吐出所有變數的新值陣列與舊值陣列 ([newFirst, newLast], [oldFirst, oldLast])，方便在內部進行交叉比對。

04-WatchEffect

自動依賴追蹤的監聽機制

•**自動偵測（免指定目標）**：前半段範例中，你不需要寫 `watch([a, b], ...)`。Vue 就像錄影機一樣，在第一次執行時發現你拿了 `a.value` 和 `b.value` 來計算，它就會自動幫你盯緊這兩個變數。

•**立即啟動**：不需要額外設定配置項，它天生在畫面初次渲染時就會先跑一次，以便收集它需要監聽哪些依賴。

•**動態依賴收集（聰明的效能優化）**：後半段 c 的範例說明了 Vue 的智慧機制。它每次執行時，只會追蹤當下真正走到的程式碼分支。如果某些變數藏在沒被執行的 `if` 或 `else` 區塊內，Vue 就不會浪費效能去監聽它們，直到路線切換為止。

比較維度	watch (第一～三組)	watchEffect (第四組)
監聽目標定義	手動指定 🔍 (明確寫出要看哪一個變數或 Getter)	自動追蹤 📺 (函式內用到誰，就自動監聽誰)
新舊值獲取	可以 ○ (Callback 參數自帶 <code>newVal</code> 與 <code>oldVal</code>)	不行 ✖ (拿不到舊值，只會重新執行)
初始執行時機	預設不執行 ⏸ (要等數據變動，或加 <code>immediate: true</code>)	首次立即執行 🚀 (掛載時一定先跑一次，用來收集依賴)
依賴收集機制	靜態固定 📌 (你指定看誰，它就一輩子只看誰)	動態調整 🔄 (依據 <code>if-else</code> 當下執行到的分支調整)
物件深層監聽	需手動開啟 🛠 (需視情況手動配置 <code>{ deep: true }</code>)	自動深入監聽 🗺 (只要程式碼內有點出物件屬性就會追蹤)
執行副作用	適合在特定變數改變時，執行特定操作 (如打 API)	適合讓多個狀態同步 (如 A+B 改變時，自動更新 C)

核心功能拆解

1. 父元件 `HooksDemo.vue`

- **掌控生死**：利用 `v-if="isShow"` 來控制子元件的掛載（顯示）與卸載（消失）。
- **狀態切換**：點擊「卸載/掛載子元件」按鈕時，會切換 `isShow` 的布林值，藉此觸發子元件的生命週期。

2. 子元件（`ChildWithHooks.vue`）

- **監聽生命週期**：匯入了 Vue 3 最常用的 6 個生命週期鉤子：
 - **掛載階段**：`onBeforeMount`（DOM 掛載前）→ `onMounted`（DOM 掛載後，可操作實體 DOM）。
 - **更新階段**：點擊 `+1` 按鈕改變 `counter` 時，觸發 `onBeforeUpdate`（畫面更新前）→ `onUpdated`（畫面更新後）。
 - **銷毀階段**：當父元件將其卸載時，觸發 `onBeforeUnmount`（卸載前，清理定時器或事件）→ `onUnmounted`（已完全卸載）。
- **視覺化紀錄**：內建 `log` 函式，除了在 Console 印出訊息，也把訊息塞進 `events` 陣列，直接渲染在畫面上供開發者對照。

01-Hooks資料夾
`HooksDemo.vue`
`ChildWithHooks.vue`

您可以觀察到的現象

1. **首次載入（或點擊掛載）**：畫面上會依序噴出 `onBeforeMount` 與 `onMounted` 的紀錄。
2. **點擊 +1 按鈕**：畫面上會依序增加 `onBeforeUpdate` 與 `onUpdated` 的紀錄。
3. **點擊卸載子元件**：子元件從畫面上消失。此時你必須打開瀏覽器 **DevTools Console**，才會看到最後落下的 `onBeforeUnmount` 與 `onUnmounted` 紀錄（因為此時子元件的畫面已經不見了）。

示範為什麼組件被銷毀 (Unmount) 時，一定要清除計時器 (setInterval)，以及不清除會帶來什麼嚴重的後果。

1. 父組件 (外層控制)

負責控制子組件的「掛載 (顯示)」與「卸載 (消失)」。

- **核心機制**：使用 `v-if="isRunning"` 來動態切換 `TimerCore` (子組件) 的存在。
- **互動設計**：當使用者點擊「卸載計時器」按鈕時，`isRunning` 變為 `false`，此時子組件會被完全從 DOM 樹中移除 (觸發 Unmount 流程)。

2. 子組件 `TimerCore.vue` (計時器核心)

負責執行計時功能，並示範正確的資源清理流程。

- **掛載時 (`onMounted`)**：組件畫面出現時，啟動 `setInterval`，每秒讓 `seconds` 加 1，並把計時器的 ID 存到 `intervalId` 變數中。
- **卸載時 (`onUnmounted`)**：當父組件將其移除時，主動執行 `clearInterval(intervalId)` 來關閉計時器。

02-PracticalTimer資料夾
`PracticalDemo.vue`
`TimerCore.vue`

展示 Options API (舊寫法) 與 Composition API (新寫法) 在生命週期鉤子 (Lifecycle Hooks) 上的對應關係。

1. 父組件 (新寫法 `Composition API`)

作為對照組的外層，負責控制與展示。

- **展示對照表**：直接在畫面上列出新舊語法的 8 個生命週期對照。
- **開關子組件**：利用 `v-if="isShow"` 控制舊寫法子組件的掛載與卸載，讓開發者可以在瀏覽器的 Console 中看到舊組件的生命週期觸發順序。

2. 子組件 `OptionsApiChild.vue` (舊寫法 `Options API`)

完全使用 Vue 2 時代延續下來的舊語法撰寫，用來實際觸發這 8 個生命週期。

- **初始化階段**：執行 `beforeCreate` 與 `created`，並在畫面的列表中記錄觸發順序。
- **畫面互動**：當使用者點擊 `+1` 按鈕時，會觸發 `beforeUpdate` 與 `updated`。
- **銷毀階段**：當父組件按鈕被點擊、子組件消失時，會觸發 `beforeUnmount` 與 `unmounted`。

這個範例想傳達的 2 個核心觀念

核心 1：數量從 8 個變成 6 個 (最重要差異)

- **舊寫法 (Options API)** 有 8 個常用階段。
- **新寫法 (Composition API)** 只有 6 個。因為舊的 `beforeCreate` 和 `created` 這兩個階段，在新寫法的 `<script setup>` 中不需要了。你在 `<script setup>` 裡直接寫的程式碼，就等同於舊寫法的 `created` 階段。

核心 2：命名規則的改變

除了前兩個被整合之外，其餘 6 個核心階段功能完全相同，新寫法只是在前面統一加上了 `on`，並改成小駝峰命名。例如：

- `mounted` 變成 `onMounted`
- `unmounted` 變成 `onUnmounted`

Single-File Component(SFC)

- 副檔名為 .vue
- 包含 <template>、<script> 和 <style>

使用元件

```
<script setup>
import Test from './components/Test.vue'
</script>

<template>
  <Test />
</template>
```

定義元件時

- 使用 PascalCase (駝峰式命名)
- 每個單字首字母大寫
- 例如：UserProfile、TodoList、NavigationBar

```
components/
├─ UserProfile.vue  ✓ 推薦
├─ user-profile.vue  ✓ 也可以
├─ userProfile.vue   ✗ 不推薦
└─ user_profile.vue  ✗ 不推薦
```

在 template 中使用時

- 可以使用 PascalCase：<UserProfile />
- 也可以使用 kebab-case：<user-profile />
- HTML 標籤不分大小寫，所以都會被轉換為小寫

```
<template>
  <!-- 兩種寫法都可以 -->
  <UserProfile />      <!-- ✓ PascalCase -->
  <user-profile />    <!-- ✓ kebab-case -->

  <!-- 不推薦 -->
  <userProfile />     <!-- ✗ camelCase -->
  <User_Profile />   <!-- ✗ 其他格式 -->
</template>
```

父傳子：prop

11-component資料夾

父元件BlogList.vue

傳給子元件BlogPost.vue

子傳父：emit

什麼是 emit？

emit 是 Vue 中用於子元件向父元件傳遞事件和資料的機制。當我們需要讓子元件「通知」父元件某些事情發生時，就會使用 emit。

11-component資料夾
子元件CounterButton.vue
傳給父元件Counter.vue

父傳子：prop

子傳父：emit

這兩種比較適合單純只有父子時

Vueuse

<https://vueuse.org/core/useDraggable/>

在15-vueuse資料夾的components下建立Demo.vue檔案
複製程式碼貼到Demo.vue

The screenshot shows the VueUse documentation page for the `useDraggable` function. The left sidebar lists various VueUse functions, with `useDraggable` highlighted. The main content area shows the usage code for `useDraggable` in a Vue component. A red checkmark is drawn over the code. To the right of the code, there is a live demo showing three draggable boxes: "Drag me!" (position 904, 63), "Renderless component" (position 954, 127), and "Drag here!" (position 964, 211). A fourth box, "Not Use Captured", shows that dragging it does not trigger the event (position 1063, 291).

VueUse

Search Ctrl K

Guide ☐ Functions ☐ Resources ☐ Playground ☐ v14.3.0

Check the floating boxes

Usage

```
<script setup lang="ts">
import { useDraggable } from '@vueuse/core'
import { useTemplateRef } from 'vue'

const el = useTemplateRef('el')

// `style` will be a helper computed for `left: ?px; top: ?px;`
const { x, y, style } = useDraggable(el, {
  initialValue: { x: 40, y: 40 },
})
</script>

<template>
<div ref="el" :style="style" style="position: fixed">
  Drag me! I am at {{ x }}, {{ y }}
</div>
</template>
```

Drag me!
I am at 904, 63

Renderless component
Position persisted in sessionStorage
954, 127

Drag here!
Handle that triggers the drag event
I am at 964, 211

Not Use Captured
Dragging here will not v
1063, 291

記得去App.vue匯入
`import Demo from './components/Demo.vue';`
並在<template>裡面寫 `<Demo />`
`npm run dev`就會看到黃色這個可拖拉的東西

15 — VueUse 實戰

Drag me! I am at 848.666748046875, 58.64582443237305

請對照 README.md 一起閱讀。第 4 個範例 (useDark) 會切換整個頁面的配色。

1. 你在第 14 章寫過的，VueUse 都有現成的

第 14 章你親手實作了 `useMouse` 跟 `useFetch`，就是為了理解 `composable` 的內部運作。實務上：**VueUse** 已經幫你寫好了，而且處理了更多 *edge case*。

useMouse()

滑鼠座標：x = **677**, y = **295**

```
import { useMouse } from '@vueuse/core'

const { x, y } = useMouse()
```

useFetch()

送出請求

2. useLocalStorage

跟你在第 14 章寫的概念一樣，但 **VueUse** 版本還幫你處理：JSON 自動序列化、預設值合併、跨 tab 同步、SSR 安全。

姓名：

pinia

```
PS C:\Users\Galaxy\Desktop\vue-course\18-pinia> cd ..
PS C:\Users\Galaxy\Desktop\vue-course> npm create vue@latest

> npx
> create-vue

  Vue.js - The Progressive JavaScript Framework
  ◇ 請輸入專案名稱：
    pinia-demo
  ◇ 是否使用 TypeScript？
    No
  ◇ 請選擇要包含的功能：（↑/↓ 切換，空格選擇，a 全選，enter 確認）
    Pinia（狀態管理）
  ◇ 請選擇要包含的試驗特性：（↑/↓ 切換，空格選擇，a 全選，enter 確認）
    none
  ◇ 跳過所有範例程式碼，建立一個空白的 Vue 專案？
    No

正在建置專案 C:\Users\Galaxy\Desktop\vue-course\pinia-demo...

專案建置完成，可執行以下命令：

cd pinia-demo
npm install
npm run dev
```

```
package.json X
18-pinia > package.json > ...
bucky0112, 2 週前 | 1 author (bucky0112)

1  {
2    "name": "18-pinia",
3    "version": "0.0.0",
4    "private": true,
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "dependencies": {
12     "pinia": "^3.0.4", ✓
13     "pinia-plugin-persistedstate": "^4.7.1",
14     "vue": "^3.5.13"
15   },
16   "devDependencies": {
17     "@vitejs/plugin-vue": "^5.2.3",
18     "vite": "^6.2.4",
19     "vite-plugin-vue-devtools": "^7.7.2"
20   }
21 }
22
```

使用pinia的話可以開F12的小鳳梨來看狀態

來自 pinia 的 Demo 目前記數: 0

雙倍記數: 0

加1

Component A

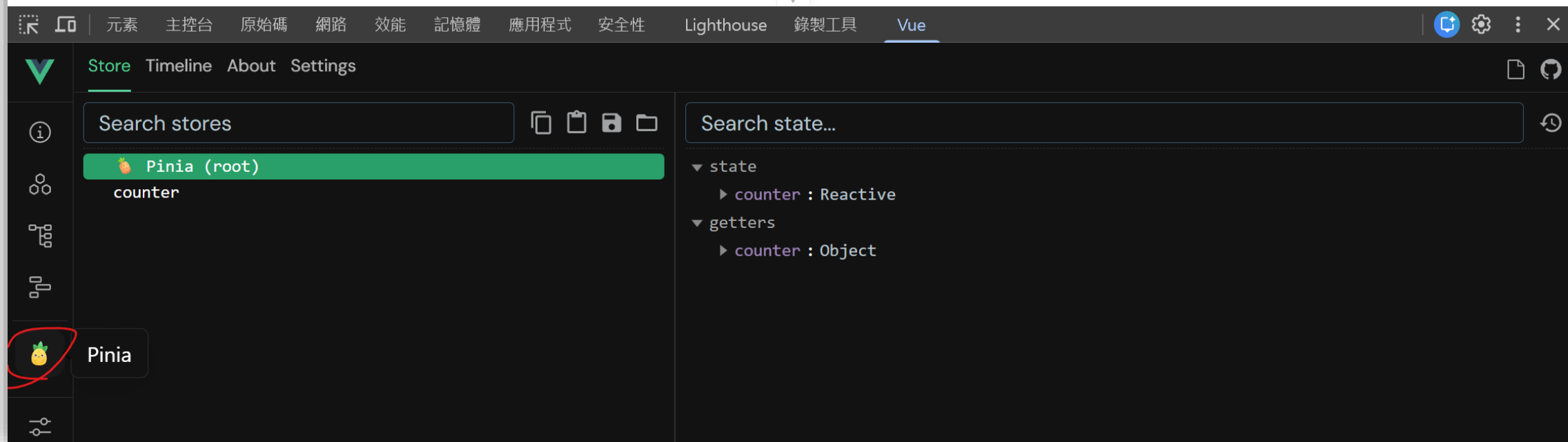
目前計數: 0

Component B

目前計數: 0

Component C

加1



The screenshot displays the Vue DevTools interface, specifically the Pinia state management tool. The top bar features tabs for Store, Timeline, About, and Settings. The left sidebar contains a search bar labeled "Search stores" and a list of stores, with "Pinia (root)" selected. The right sidebar shows a search bar labeled "Search state..." and a tree view of the state, including "state" and "getters".

Search stores

Pinia (root)

counter

Search state...

state

- counter : Reactive

getters

- counter : Object

router

```
• PS C:\Users\Galaxy\Desktop\vue-course\19-router> cd ..
• PS C:\Users\Galaxy\Desktop\vue-course> npm create vue@latest

> npx
> create-vue

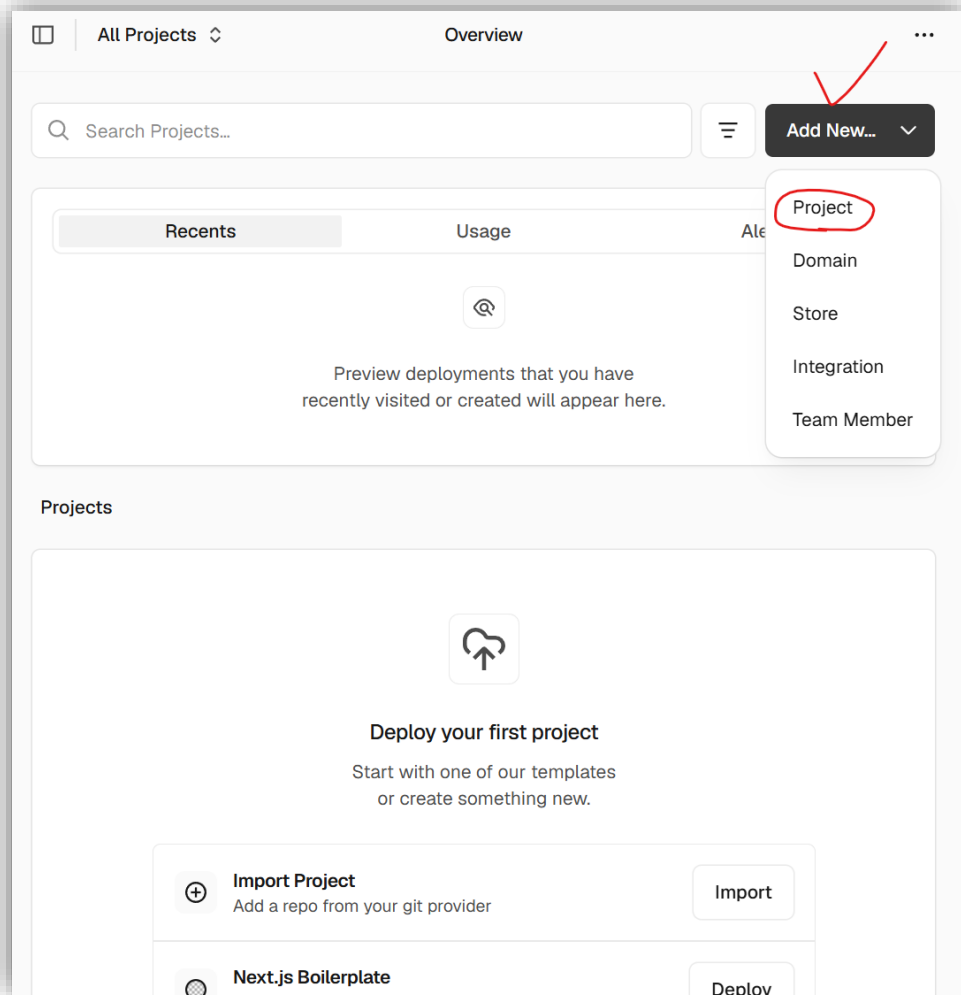
  Vue.js - The Progressive JavaScript Framework
  ◇ 請輸入專案名稱：
    router-demo
  ◇ 是否使用 TypeScript？
    No
  ◇ 請選擇要包含的功能：（↑/↓ 切換，空格選擇，a 全選，enter 確認）
    Router（單頁應用程式開發）
  ◇ 請選擇要包含的試驗特性：（↑/↓ 切換，空格選擇，a 全選，enter 確認）
    none
  ◇ 跳過所有範例程式碼，建立一個空白的 Vue 專案？
    No

正在建置專案 C:\Users\Galaxy\Desktop\vue-course\router-demo...

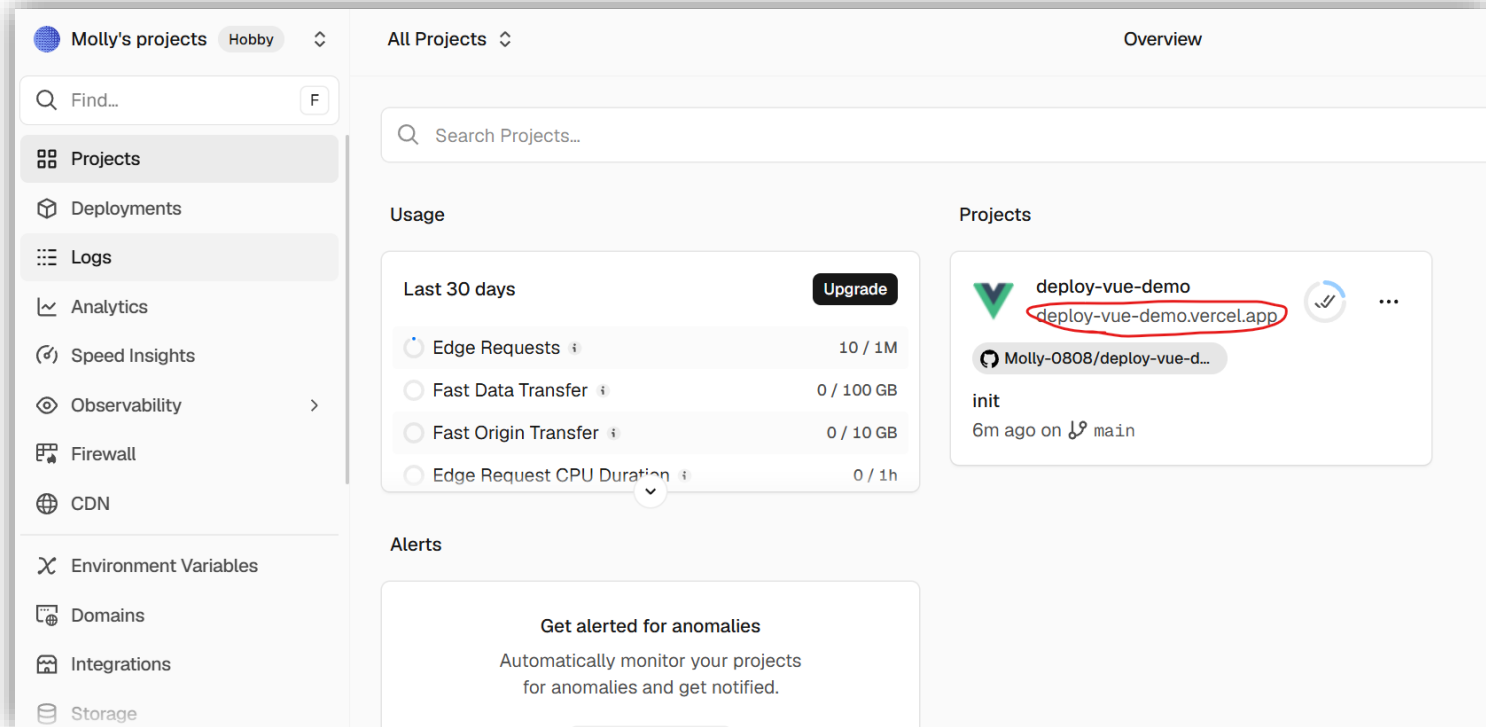
  專案建置完成，可執行以下命令：

  cd router-demo
  npm install
  npm run dev
```

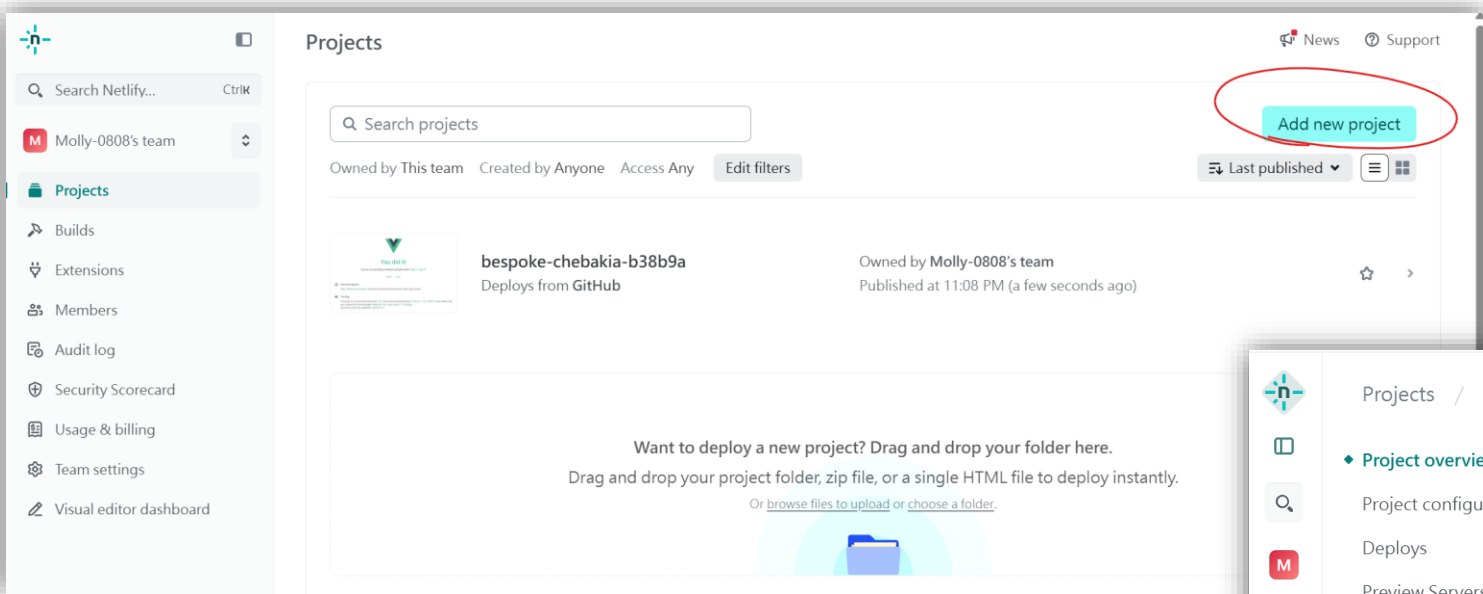
前端部屬(vercel)



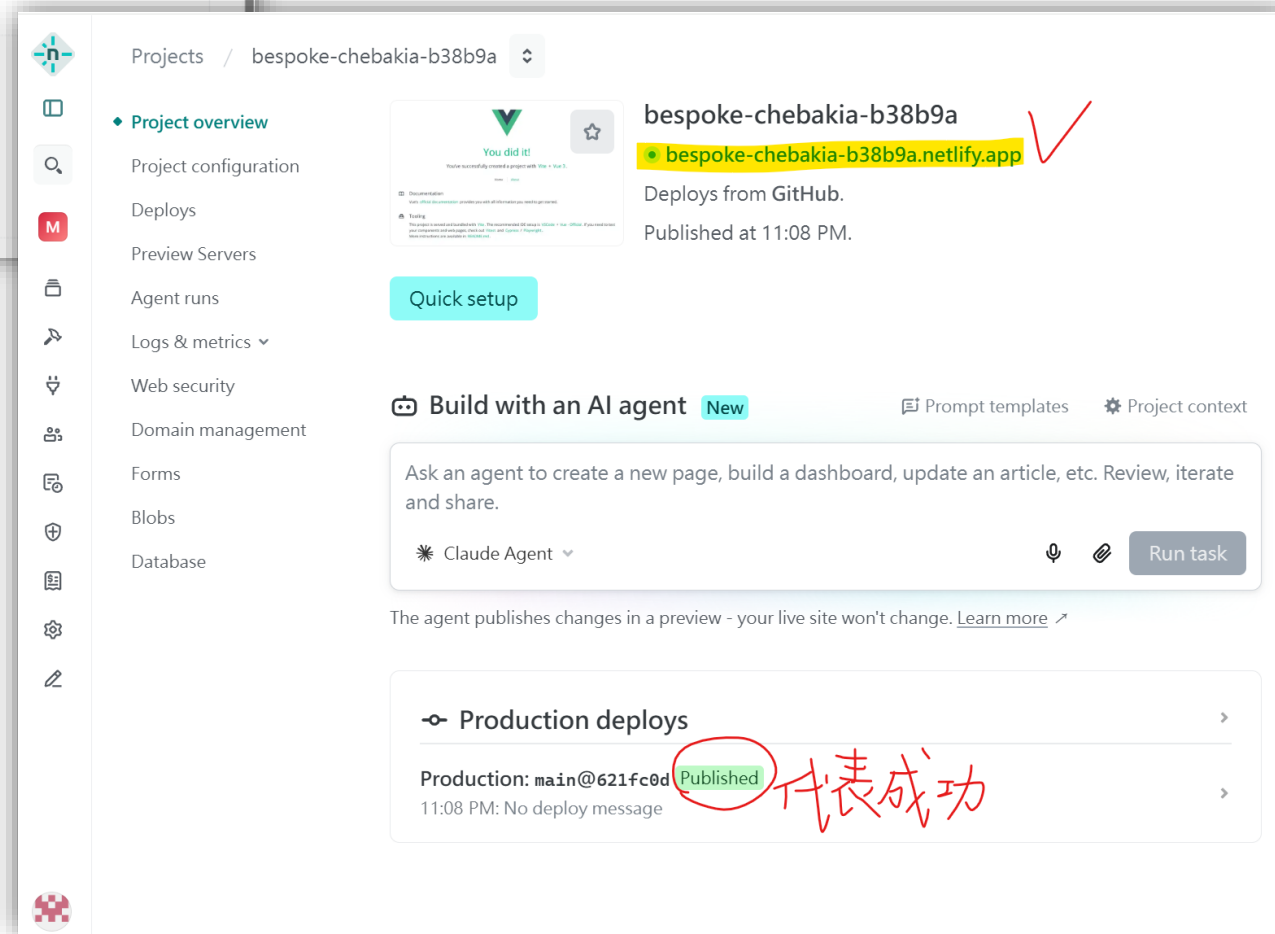
這個網址可以看我專案長怎樣



前端部屬(netlify)



這個網址可以看我專案長怎樣



Install Tailwind CSS

<https://tailwindcss.com/docs/installation/using-vite>

02

Install Tailwind CSS

Install `tailwindcss` and `@tailwindcss/vite` via npm.

Terminal

```
npm install tailwindcss @tailwindcss/vite
```

問題

輸出

偵錯主控台

終端機

連接埠

GITLENS

PS C:\Users\Galaxy\Desktop\deploy-vue-demo> npm install tailwindcss @tailwindcss/vite

貼在終端機(記得先確認路徑)

Install Tailwind CSS

<https://tailwindcss.com/docs/installation/using-vite>

把亮起來的地方新增到我的專案的vite.config.js裡面

03

Configure the Vite plugin

Add the `@tailwindcss/vite` plugin to your Vite configuration.

vite.config.ts

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

Install Tailwind CSS

<https://tailwindcss.com/docs/installation/using-vite>

複製貼到assets資料夾裡面的main.css

04

Import Tailwind CSS

Add an `@import` to your CSS file that imports Tailwind CSS.

CSS

```
@import "tailwindcss";
```

```
main.css M X
src > assets > main.css > ...
You, 51 分鐘前 | 1 author (You)
1 @import "./base.css";
2 @import "tailwindcss"; ✓
3
4 #app {
5   max-width: 1280px;
6   margin: 0 auto;
7   padding: 2rem;
```

然後就可以npm run dev了

Install Tailwind CSS

<https://tailwindcss.com/docs/installation/using-vite>

接著就可以去HTML檔案寫Tailwind了

06

Start using Tailwind in your HTML

Make sure your compiled CSS is included in the ``<head>`` (your framework might handle this for you), then start using Tailwind's utility classes to style your content.

HTML

```
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-s
  <link href="/src/style.css" rel="stylesheet">
</head>
<body>
  <h1 class="text-3xl font-bold underline">
    Hello world!
  </h1>
</body>
</html>
```

示範Tailwind語法